

Guião de Demonstração — Gestor de Cronograma FORAVE

Este documento descreve, passo a passo, como demonstrar as funcionalidades do sistema. Está organizado por **perfil de utilizador** (Coordenador, Professor e Aluno), numa sequência lógica: o Coordenador prepara o curso, o Professor gere o seu módulo e lança notas, e o Aluno consulta os seus resultados.

Para além de *o que* fazer, o guião explica também *porque* as coisas estão feitas como estão — para dar uma noção da estratégia e das decisões de engenharia à medida que se percorre a demonstração. Cada passo usa três registos visuais:

- **Passo numerado** — a ação a realizar (o que clicar/escrever).

Resultado esperado

O que se deve ver no ecrã depois da ação (caixa verde).

Porquê / RGPD / Nos bastidores

Comentário: a decisão de design, a escolha de privacidade ou um detalhe de como funciona por baixo (caixa azul). É opcional — pode ler-se ou saltar-se sem perder o fio dos passos.

Contas para a demonstração

Perfil	Email	Password
Coordenador	chronosforave@gmail.com	demo1234
Professor	luis.cerejeira@forave.pt	demo1234
Alunos	sofia.marques@forave.pt tiago.fernandes@forave.pt ines.rocha@forave.pt	demo1234
Aluno de teste (email real)	alunoforavectdados@gmail.com	demo1234

Estas contas e a password `demo1234` são criadas pelo `python seed_demo.py` e servem a demonstração **local** (terminal, web local e `.exe`). O **aluno de teste** tem um email real: é nele que se recebem os avisos automáticos por email.

No **site alojado** (secção 6), o acesso de coordenador é feito com as credenciais que acompanham a entrega (indicadas à parte, nunca no código).

Índice

Guião de Demonstração — Gestor de Cronograma FORAVE

- 0. Preparação
- 1. Página pública (sem sessão iniciada)
- 2. Perfil: Coordenador
 - 2.1 Visão geral
 - 2.2 Módulos (criar, importar, editar, remover)
 - 2.3 Professores (duas formas)
 - 2.4 Alunos (inscrever, importar, editar, remover)
 - 2.5 Cronograma — agendar as aulas (data, horas e "aula dada")
 - 2.6 Exportação e auditoria
- 3. Perfil: Professor
- 4. Perfil: Aluno
- 5. Notificações por email
- 6. Outras interfaces e canais
 - 6.1 Terminal (`python main.py`)
 - 6.2 Executável (`.exe`) — o terminal sem precisar de Python
 - 6.3 Site alojado
 - 6.4 Instalação como aplicação (PWA)
- 7. Reposição de password

- 8. Estrutura do código (para consulta)
- 9. Alinhamento com a matéria de Python Avançado
- Resumo das funcionalidades

0. Preparação

1. Abrir a pasta do projeto no editor de código (VS Code).
2. Instalar as dependências (se necessário): `pip install -r requirements.txt`

Carregar os dados de demonstração (módulos, professores, alunos, avaliações, notas e contas de acesso): `python seed_demo.py`

Resultado esperado

Mensagem de conclusão com o número de módulos, alunos e contas criados.

Nos bastidores

O `seed_demo.py` é **idempotente**: só cria dados de exemplo e pode correr as vezes que se quiser sem duplicar nem apagar nada. Isto torna a demo repetível — se algo correr mal a meio, basta voltar a semear.

A aplicação pode ser demonstrada em três ambientes:

Ambiente	Comando de arranque	Endereço
Web (local)	<code>python app.py</code>	<code>http://127.0.0.1:5000</code>
Terminal	<code>python main.py</code>	—
Site alojado	(já em execução)	URL do alojamento (ver secção 6)

Porquê

O mesmo domínio (as classes em `classes/` e o `gestor_cronograma.py`) serve os três ambientes — só muda a "casca" que o apresenta. É a prova prática da separação entre **lógica** e **apresentação**: a regra de negócio escreve-se uma vez e reutiliza-se no terminal, na web e no executável.

1. Página pública (sem sessão iniciada)

Abrir `http://127.0.0.1:5000` sem iniciar sessão.

Resultado esperado

Página inicial com a identidade do curso (CET, UFCD 10794, professor e grupo de trabalho).

Localizar a secção "Leva o cronograma para o teu calendário" (botão "Adicionar ao calendário (.ics)").

Resultado esperado

Botão de download .ics e um código QR do cronograma.

RGPD

O horário é assumido como informação **pública** — não são dados pessoais — por isso o .ics do curso está aberto. Os dados pessoais (as notas) **nunca** aparecem nesta página; ficam sempre atrás de autenticação. Separámos de propósito o que é público do que é privado.

Introduzir um endereço inexistente (ex.: `/xyz`).

Resultado esperado

Página de erro **404** com a identidade visual do sistema.

Nos bastidores

Até as páginas de erro (403/404/500) têm a marca e são tratadas pela aplicação. Somam-se a cabeçalhos de segurança HTTP aplicados a todas as respostas — pequenos toques de "produto", não só de "trabalho de escola".

2. Perfil: Coordenador

Iniciar sessão com `chronosforave@gmail.com` / `demo1234`.

Resultado esperado

Acesso à área de Administração, com visão de todos os módulos.

Porquê

O perfil (coordenador / professor / aluno) é determinado pelo **email** com que se inicia sessão, não por um campo à parte que se possa contradizer. Uma única fonte de verdade para a identidade evita incoerências.

2.1 Visão geral

Consultar o painel principal.

Resultado esperado

Cartões de resumo, um **gráfico de progresso dos módulos** (% concluído, ordenado dos mais atrasados para os mais adiantados), lista de módulos e secção de indicadores.

Nos bastidores

Os indicadores e o gráfico são **calculados** a partir dos dados reais no momento — não são imagens fixas. O gráfico responde a uma pergunta útil ao coordenador ("quais os módulos atrasados?"), em vez de uma contagem sem sinal.

2.2 Módulos (criar, importar, editar, remover)

Em Adicionar módulo, preencher o nome, a UFCD, o professor, as horas totais previstas e o estado, e submeter.

Resultado esperado

O módulo passa a constar na lista de módulos.

Porquê

Não se pede aqui "horas dadas": um módulo novo ainda não teve aulas, logo começa a **0**. As horas dadas passam a ser **calculadas** à medida que as aulas são marcadas (ver 2.5) — não é um número escrito à mão que possa divergir da realidade.

Em Importar módulos por CSV, seleccionar `demo_modulos.csv` e importar.

Resultado esperado

Mensagem "Importação: N importado(s)". Módulos já existentes são assinalados como não importados.

Nos bastidores

A importação **deteta duplicados** pela chave do módulo (o nome): reimportar o mesmo ficheiro não cria repetidos, apenas assinala os que saltou. Oferecer as duas formas — manual e CSV — cobre

tanto a edição pontual como a carga rápida de um curso inteiro exportado do Excel.

Editar um módulo (professor, UFCD, estado, horas totais).

Resultado esperado

Os valores atualizados refletem-se na lista.

Porquê

As horas **dadas** não se editam aqui — o campo é só de leitura, porque são calculadas a partir das aulas marcadas no cronograma (ver 2.5). Manter uma só forma de as alterar evita dois números a competir pela verdade.

Abrir um módulo e utilizar Remover módulo (confirmar no diálogo).

Resultado esperado

O módulo, com as suas avaliações e notas, é eliminado.

RGPD

A remoção é **em cascata**: apagar o módulo leva as avaliações e notas associadas, sem deixar dados órfãos. A confirmação existe de propósito — é uma ação destrutiva e irreversível.

2.3 Professores (duas formas)

Em Professores -> Adicionar, indicar nome, email e módulos que gere.

Resultado esperado

O professor passa a constar na lista.

Porquê

Associar um email a um professor é o que lhe **atribui o perfil** de Professor ao iniciar sessão (ver secção 3). O email é a ligação entre a pessoa e aquilo que está autorizada a ver.

Em importar professores por CSV, seleccionar `demo_professores.csv`.

Resultado esperado

Professores importados; mensagem de resultado.

Editar ou remover um professor.

Resultado esperado

A alteração reflete-se na lista.

2.4 Alunos (inscrever, importar, editar, remover)

Em Formandos -> Inscrever, indicar nome, email e módulos.

Resultado esperado

O aluno passa a constar na lista.

Em importar formandos por CSV, seleccionar `demo_formandos.csv`.

Resultado esperado

Alunos importados; mensagem de resultado.

Abrir um aluno e editar o nome ou os módulos inscritos; guardar.

Resultado esperado

Os dados atualizam-se.

Porquê

O **email** é a identidade do aluno — é a chave a que as notas estão associadas — por isso não se edita. Para o mudar, remove-se e volta-se a inscrever. Alterar uma chave em uso é sempre uma operação delicada; preferimos torná-la explícita.

Remover um aluno.

Resultado esperado

Surge um pedido de confirmação; ao confirmar, o aluno e as suas notas são eliminados.

RGPD

Apagar o aluno leva **também** as suas notas — o *direito ao apagamento* não fica meio feito. A confirmação prévia protege contra remoções acidentais.

2.5 Cronograma — agendar as aulas (data, horas e "aula dada")

Abrir um módulo e, na secção Cronograma (aulas), indicar o número de aulas e gerar os campos. Cada aula tem três campos: a data (escolhida num calendário), as horas

dessa aula e uma marca "dada" (). Guardar.

Resultado esperado

Os dias de aula ficam agendados. As **horas dadas** do módulo passam a ser a **soma das horas das aulas marcadas como dadas** — o indicador "X/Y h" e as "horas em falta" atualizam-se sozinhos.

Porquê

As horas dadas são **calculadas**, não escritas à mão. Uma só fonte de verdade (as aulas marcadas) alimenta o indicador de progresso e as horas em falta — nunca há um total manual a contradizer o cronograma.

Nos bastidores

Cada aula (data + horas + "dada") é guardada numa coluna **"Sessoes"** (em JSON) na Google Sheet, de forma **retrocompatível**: módulos antigos, que só tinham datas e um total escrito à mão, continuam a funcionar. Guardar o cronograma **não apaga** um total já importado enquanto não se marcar nenhuma aula — os módulos vindos de CSV estão protegidos.

Marcar mais uma aula como dada () e guardar de novo.

Resultado esperado

As horas dadas aumentam e as horas em falta diminuem, sem qualquer edição manual do número.

Módulos importados por CSV

O CSV traz as horas **totais** e as datas das aulas, mas **não** as horas de cada aula nem a marca "dada" — é aqui, no cronograma, que essas se definem à medida que o curso avança. (Decisão consciente: pôr horas por sessão no CSV seria uma segunda porta de entrada para o mesmo dado, a arriscar divergências.)

2.6 Exportação e auditoria

Gerar o Relatório PDF.

Resultado esperado

Download de um PDF com módulos e avaliações (com indicador de progresso durante a geração).

Utilizar a exportação .ics (curso completo) e, num módulo, .ics + QR.

Resultado esperado

Ficheiro de calendário / código QR do módulo.

Porquê

O QR só é usado onde acrescenta algo: como ponte de um meio **não clicável** (um cartaz, um slide) para o telemóvel. Na própria página não se põe um QR para a própria página — seria redundante.

Consultar o Registo de alterações e Descarregar o registo completo (CSV).

Resultado esperado

Histórico das operações (autor, ação, data) — no ecrã as mais recentes, e o **CSV com o histórico completo** para arquivo/auditoria externa. Nas notas, é apresentado o valor anterior e o novo.

Accountability

O registo de auditoria responde a "quem fez o quê e quando". Guardar o valor **anterior** -> **novo** nas notas dá rastreabilidade real — importante porque a autorização é por perfil, e o registo é a rede de segurança que a acompanha.

3. Perfil: Professor

Terminar sessão e iniciar com `luis.cerejeira@forave.pt` / `demo1234`.

Aceder à Área do Professor ("Os meus módulos"). No topo, um **Calendário com as aulas e avaliações dos seus módulos** (mesma peça da área do aluno, agora filtrada ao âmbito do professor), com o botão **.ics** e o **QR** para levar o cronograma para o telemóvel.

Resultado esperado

É apresentado **apenas** o módulo atribuído (*Programação Avançada com Python*); os restantes módulos não são visíveis. O calendário mostra só os eventos desse módulo.

Porquê

A autorização é **por âmbito**: o professor vê e gere só os seus módulos, não é tudo-ou-nada. Foi uma evolução deliberada de um controlo de acesso simples (só coordenador podia tudo) para permissões mais finas por módulo.

Accountability

Tudo o que o professor altera (notas, cronograma, avaliações) fica no **registo de auditoria** com o **email dele** como autor. O professor é autónomo no seu módulo, mas nada do que faz fica sem rasto — o coordenador vê o histórico completo (o professor não vê a auditoria global).

Confirmar a ausência das secções de coordenação (criar módulos, gerir professores e formandos, auditoria global).

Resultado esperado

Estas secções não estão disponíveis para o professor.

No Cronograma (aulas) do módulo, marcar como dada () uma aula ainda por dar e guardar.

Resultado esperado

As **horas dadas** do módulo aumentam (soma das aulas dadas) e as horas em falta diminuem.

Porquê

O professor gere as horas do seu módulo **sem depender do coordenador** — a autorização por âmbito torna-o autónomo dentro do que é seu.

Criar ou editar uma avaliação no módulo. A data escolhe-se num calendário; cada campo tem uma dica do que se espera (só a *Descrição* é obrigatória). Opcionalmente, marcar "Avisar a turma por email".

Resultado esperado

A avaliação passa a constar; o campo "tipo" apresenta sugestões (Teste, Projeto, etc.). Com a caixa marcada e uma data definida, cada aluno inscrito recebe um email a anunciar (ou atualizar) a avaliação.

Nos bastidores

As sugestões vêm de uma lista (data list) para uniformizar os tipos sem obrigar a menus rígidos. O aviso à turma reutiliza o mesmo mecanismo do reagendamento de aulas (envio individual a cada aluno, nunca em conjunto).

Lançar uma nota a um aluno. Para demonstrar a notificação individual, marcar a opção "avisar cada aluno da sua nota por email" antes de guardar — utilizar o aluno de teste `alunoforavecetdados@gmail.com` para verificar a receção (secção 5).

Resultado esperado

A nota é registada; com a opção marcada, é enviado ao aluno um email com a sua nota. A nota fica visível na área do aluno (secção 4).

RGPD

O aviso de nota é **individual**: cada aluno é notificado apenas da **sua** nota, nunca das dos outros. A notificação respeita o mesmo isolamento de dados que o resto do sistema.

Reagendar uma aula (alterar uma data) com a opção "avisar a turma" e um motivo.

Resultado esperado

A data é atualizada e é preparada a notificação à turma (ver secção 5).

Porquê

Este era o problema real que deu origem ao projeto: as mudanças de data não chegavam a tempo aos formandos. Aqui, reagendar e avisar a turma é **uma só ação** — a comunicação deixa de depender de alguém se lembrar de a fazer.

4. Perfil: Aluno

Terminar sessão e iniciar com `sofia.marques@forave.pt / demo1234`. O aluno entra na sua página pessoal, "**A minha área**" (calendário + notas).

Consultar "O meu calendário" no topo da página. Alternar entre Mês e Semana e navegar com < > e Hoje.

Resultado esperado

Um calendário com as **aulas** (verde; as já dadas a verde-suave com) e as **avaliações** (dourado) dos módulos do aluno. O dia de hoje fica destacado.

Nos bastidores

O calendário é **feito de raiz** (`static/calendario.js`), sem bibliotecas externas — só JavaScript e o DOM. O servidor prepara os eventos **só deste aluno** (RGPD); o cliente desenha as vistas mês/semana.

Por baixo do calendário, usar "Adicionar ao meu telemóvel (.ics)" ou o QR code.

Resultado esperado

Descarrega um ficheiro `.ics` (ou, pelo QR, abre-o no telemóvel) que junta as aulas ao calendário do dispositivo.

Porquê

O QR é a ponte ecrã->telemóvel: reutiliza o `.ics` **público** do curso (o horário não são dados pessoais — as notas nunca saem por aqui).

Consultar As minhas notas.

Resultado esperado

Módulos em que está inscrita, cada avaliação com a nota respetiva e a **nota final (UFCD)** calculada por média ponderada (ex.: **16.9** em Programação Avançada).

Nos bastidores

A nota final é uma **média ponderada** pelos pesos das avaliações, calculada a partir das notas lançadas — o aluno vê o resultado sem ter de fazer contas.

Verificar a nota lançada pelo professor na secção 3.

Resultado esperado

A nota está visível para a aluna.

Confirmar o isolamento de dados.

Resultado esperado

A aluna não tem acesso a notas de outros alunos.

RGPD

O isolamento de dados é um requisito, não um detalhe: cada aluno vê apenas o que é seu. É a mesma regra que atravessa a página pública, as notificações e a área pessoal.

(Opcional) Iniciar sessão como `tiago.fernandes@forave.pt`.

Resultado esperado

Apenas os dados desse aluno são apresentados.

5. Notificações por email

O sistema envia avisos por email em três casos: **nota individual** ao aluno, **reagendamento** de uma aula à turma e **avaliação marcada/atualizada** à turma.

1. Lançar uma nota (ou reagendar uma aula) que envolva o aluno de teste `alunoforavecetdados@gmail.com`.

Verificar a caixa de correio desse aluno, incluindo a pasta de Spam.

Resultado esperado

Receção do aviso correspondente. O email vem de "**Cronograma FORAVE**" (chronosforave@gmail.com), mas ao clicar em **Responder** a resposta abre para **quem fez a alteração** (ex.: o professor).

Remetente institucional, resposta pessoal (Reply-To)

O remetente é **sempre** o endereço institucional verificado na Brevo — não pode ser o email pessoal do professor (a Brevo rejeitaria um remetente não verificado). Para não perder o toque pessoal, define-se um **Reply-To** com o email de quem despoletou o aviso: o envio continua institucional (e aceite), mas as **respostas dos alunos vão ter com o professor/coordenador** que agiu.

Nos bastidores — porquê a Brevo

No alojamento (Render), o envio direto por SMTP está **bloqueado** — uma restrição comum nos serviços de alojamento para travar spam. Por isso o envio em produção passa por um **serviço transacional (Brevo)**, por API. Localmente, o envio direto por SMTP continua disponível como alternativa. A arquitetura prevê os dois caminhos.

Porquê pode ir para Spam

O remetente da demo é um endereço **gratuito** (Gmail). Pelas regras de autenticação de email (SPF/DKIM/DMARC), uma mensagem enviada em nome de um endereço gratuito através de um serviço transacional pode ser classificada como **spam** — daí a instrução de verificar essa pasta. Em produção recomenda-se um **domínio próprio**. É uma limitação conhecida e assumida, não um defeito por resolver.

Acesso à caixa de teste: por segurança, as credenciais de acesso à conta `alunoforavectdados@gmail.com` não constam do projeto; são indicadas na mensagem de entrega.

6. Outras interfaces e canais

6.1 Terminal (`python main.py`)

O terminal expõe **as mesmas operações** da versão web, sobre **os mesmos dados** (os ficheiros JSON em `dados/`), mas em menu de texto — a prova de que a regra de negócio é independente da "casca" que a mostra.

Arrancar: `python main.py`.

Resultado esperado

Ecrã de arranque ("Dados: 3 módulos, 3 professores, 4 formandos") e depois o **menu** com as opções 1–14, S e 0, e o estado no topo (`[Google Sheets: OFF] [Email: OFF]` na cópia de demonstração).

O menu cobre todo o ciclo: **1** ver cronograma · **2** módulo · **3–4** professores · **5–6** formandos · **7** registar alteração · **8** histórico · **9** registar horas · **10** importar CSV · **11** avaliação · **12** remover formando (RGPD) · **13** lançar notas · **14** gerar QR · **S** sincronizar · **0** sair. Abaixo percorrem-se as opções que **só o terminal** tem (ou que aqui se veem melhor).

Opção 1 — Ver cronograma.

Resultado esperado

Os 3 módulos de demonstração, cada um com horas dadas/totais, percentagem e estado.

Opção 9 — Registar horas leccionadas (*exclusivo do terminal*). Escolher o módulo pelo número (o menu mostra o progresso ao lado, ex. `Python Avançado – 25/50h`) e indicar as horas a somar.

Resultado esperado

As horas somam-se e reaparece o total (ex. 30h registadas. Total: 30/50h (60.0%)); ao atingir o total surge "Módulo concluído!".

Web vs terminal

Na web as horas vêm das **sessões** do cronograma (data + horas + "aula dada"); no terminal há também este registo **manual e rápido** — ambos atualizam o mesmo módulo.

Opção 7 — Registar alteração e escolher o canal (*exclusivo do terminal*). Depois de descrever a alteração a um módulo, o programa pergunta como notificar: **1. Consola** · **2. Ficheiro** · **3. Email** · **4. Todos** · **5. Não enviar**.

Resultado esperado

Consola -> a mensagem aparece no ecrã; Ficheiro -> é escrita em `dados/notificacoes.txt`; Email -> enviada aos destinatários (se configurado); Todos -> faz os três.

Porquê mostrar isto

É o mesmo objeto de **Notificação** com vários **canais** — "uma lógica, várias saídas" aplicada às notificações.

Opção 8 — Ver histórico de alterações. Lista as alterações já registadas.

Opção 14 — Gerar QR code (*exclusivo do terminal*). Pede o URL (sugere `http://127.0.0.1:5000`) e o nome do ficheiro, e grava um PNG com o QR.

Resultado esperado

QR code criado: `qr_cronograma.png` — um artefacto físico (cartaz/slide) que abre o cronograma ao apontar a câmara.

Sair (opção 0).

Cuidado ao sair (sincronização com a cloud)

A opção **S** e a saída (**0**) podem escrever os dados **locais por cima** do Google Sheet. Por isso, ao **sair**, o programa **pergunta primeiro** se quer sincronizar (por defeito **não** — basta **Enter**). Assim, uma demonstração no terminal **nunca** altera, sem querer, os dados do site. (Só se liga ao Sheet quem tiver o `credentials.json`; a cópia de demonstração/.exe corre com dados locais.)

6.2 Executável (.exe) — o terminal sem precisar de Python

Descarregar o `GestorCronograma-v1.0.zip` da Release v1.0 (GitHub -> "Releases"), extrair a pasta e duplo-clique em `GestorCronograma.exe`. (Para reconstruir a partir do código, ver `INSTRUÇÕES_EXE.md`.)

Resultado esperado

Abre o mesmo programa de terminal do 6.1, já com a demo carregada (3 módulos, professores e alunos) e sem precisar de Python — corre offline.

Repetir ali um par de operações do 6.1 — por ex. opção 1 (ver cronograma) e opção 9 (registar horas) — para mostrar que é funcional, não só um ecrã.

Nos bastidores

O .exe é gerado com **PyInstaller** (`build_exe.ps1 + cronograma.spec`): empacota o programa de terminal (`main.py`) e as bibliotecas numa pasta, com os dados de demonstração ao lado, em `dados/`.

6.3 Site alojado

Abrir o URL do alojamento e iniciar sessão como coordenador; importar `demo_modulos.csv`, `demo_formandos.csv` e `demo_professores.csv`.

Resultado esperado

Os dados ficam disponíveis no site (leitura a partir do Google Sheet).

6.4 Instalação como aplicação (PWA)

A partir do site, instalar no dispositivo.

Resultado esperado

O sistema fica acessível como aplicação.

Porquê tantos canais

Terminal, web, executável, PDF, QR, .ics e email são todos "cascas" sobre a **mesma lógica**. Mostrá-los prova que a arquitetura separa a regra de negócio da forma como é consumida — o objetivo desde o início.

7. Reposição de password

No ecrã de início de sessão, seleccionar Esqueci-me da password e indicar um email.

Resultado esperado

É gerado um link seguro (token assinado, de uso único e com validade) para definir uma nova password.

Nos bastidores — segurança

As passwords são guardadas como **hash** (num só sentido) — nem nós as vemos. Por isso "esqueci-me" resolve-se com uma **reposição**, não com uma recuperação: gera-se um link assinado e temporário, em vez de reenviar a password antiga.

8. Estrutura do código (para consulta)

Cada ficheiro inclui um cabeçalho com o seu propósito; cada classe e método têm documentação (docstrings). Percurso recomendado:

- `README.md` e `ESTRUTURA_PROJECTO.md` — arquitetura, mapa de ficheiros e decisões.
- `classes/` — domínio (POO): `modulo.py`, `formando.py`, `avaliacao_final.py`, `professor.py`, `gerador_ics.py`, `token_modulo.py`, `autenticacao.py`, entre outros.
- `gestor_cronograma.py` — orquestração do domínio (CRUD e persistência).

- `app.py` (web) e `main.py` (terminal) — interfaces sobre o mesmo domínio.
- `testes/` — testes automáticos: `python -m unittest discover -s testes`

9. Alinhamento com a matéria de Python Avançado

O projeto aplica, num problema real, os conceitos trabalhados ao longo da UFCD — partindo de Python puro (terminal + POO) e crescendo para web + cloud, sempre com a **mesma lógica** no centro.

Conceito da UFCD	Onde se aplica no projeto
Programação Orientada a Objetos (classes, atributos, métodos)	Pasta <code>classes/</code> : <code>Modulo</code> , <code>Formando</code> , <code>Professor</code> , <code>Avaliacao</code> , <code>GestorCronograma</code> , <code>Utilizador</code> ...
Responsabilidade única / separação de camadas	A lógica devolve dados (sem <code>print</code>); a apresentação fica para quem chama (<code>main.py</code> no terminal, <code>app.py</code> na web)
Dicionários e listas	Estrutura dos dados em memória; <code>to_dict()</code> / <code>from_dict()</code> para converter objeto ↔ dicionário
Ficheiros e persistência (JSON)	<code>dados/*.json</code> como fonte de verdade local (<code>stdlib json</code>)
Tratamento de erros (try/except)	Importação de CSV robusta (conta erros por linha), rede/email com <i>timeout</i> — nunca "rebenta"
Funções com parâmetros e retorno	Todos os métodos do domínio; funções puras e testáveis
Strings e f-strings	Mensagens, corpos de email, relatório PDF
Ciclos e condições	Percursos de dados (<code>for/if</code>) em todo o domínio
Modularização (multi-ficheiro)	Domínio (<code>classes/</code>) separado das interfaces (<code>app.py</code> / <code>main.py</code>) e das integrações
Integração com serviços/APIs	Google Sheets (<code>gsread</code>), email transacional (Brevo por HTTP), web (Flask)

Cada ficheiro tem um cabeçalho a explicar o seu propósito e a ligação à matéria; cada classe e método têm *docstrings*. É o percurso recomendado para o professor confirmar, no código, que cada decisão segue o que foi dado nas aulas.

Resumo das funcionalidades

- Três perfis (Coordenador, Professor, Aluno) com isolamento de dados (RGPD).
- Controlo de horas por aula: cada sessão tem horas e marca de "dada"; as horas dadas (e as horas em falta) são calculadas automaticamente.
- Múltiplos canais sobre a mesma lógica: terminal, web, PDF, QR e `.ics` / email.
- Duas formas de carregamento em massa: manual e CSV (módulos, formandos, professores).
- Execução local (terminal, web, executável) e alojada (site, PWA, Google Sheet).
- Registo de alterações (auditoria), notificações, cabeçalhos de segurança e acessibilidade.